

Deep Research Agent

Backend API and LangGraph Flow Documentation

This document explains how the Python backend works, how the LangGraph workflow moves from a user question to a final published report, and how each Python file participates in the system.

Area	Summary
Backend API	FastAPI service in api.py exposes session, run start/resume, run snapshot, health, and translation endpoints.
LangGraph Orchestration	graph.py compiles a StateGraph with guardrails, history review, planning, search, evidence selection, reasoning, synthesis, human review, publishing, and persistence nodes.
Workflow Logic	nodes.py contains the actual node implementations, helper scoring/reranking utilities, report normalization, interrupt handling, evidence selection, and history persistence actions.
Contracts	state.py defines graph state keys. schemas.py defines strict Pydantic request, response, interrupt, and LLM output models.
External Tools	tools.py creates Tavily, Wikipedia, weather, news, politics, and sports LangChain tools. graph.py allows only tools approved by guardrails.

1. High-Level Architecture

The app has three major layers. The frontend calls FastAPI. FastAPI delegates execution to helper functions in `app.py`. Those helpers invoke a compiled LangGraph application from `graph.py`. The graph stores run state by `thread_id` and pauses at human review checkpoints by using LangGraph interrupts.

Layer	Main files	Responsibility
Presentation	<code>react-frontend/</code> , <code>frontend.py</code>	React is the main UI. <code>frontend.py</code> is an optional Streamlit UI for local exploration.
API boundary	<code>api.py</code> , <code>schemas.py</code>	Receives HTTP requests, validates payloads, converts graph state to frontend-friendly snapshots, and handles translation.
Run helpers	<code>app.py</code>	Creates graph instances, builds thread config, starts/resumes runs, and reads pending interrupts or current state.
Graph orchestration	<code>graph.py</code> , <code>state.py</code>	Defines LangGraph nodes, edges, routing decisions, checkpointers, shared store, and state shape.
Business logic	<code>nodes.py</code> , <code>prompts.py</code> , <code>tools.py</code> , <code>history_store.py</code>	Implements guardrails, history review, search, reasoning, synthesis, review, publishing, persistence, prompts, tools, and JSON history storage.

Typical request path

- The frontend requests POST `/api/sessions` to create a `thread_id`.
- The frontend posts a question to POST `/api/runs/start`.
- `api.py` calls `start_research_run` from `app.py`, passing a LangGraph config containing the `thread_id`.
- The graph executes until it completes or reaches an interrupt such as history review, evidence selection, or draft approval.
- `api.py` returns a `RunSnapshotResponse` containing status, guardrails, metrics, interrupt payload, draft, search results, or final report.
- When the user chooses a decision, the frontend calls POST `/api/runs/resume`. `app.py` resumes the graph with `Command(resume=...)`.

2. FastAPI Backend

`api.py` owns the HTTP API used by the React frontend. It creates one compiled `research_app` at module load time and reuses it across requests.

Endpoint	Purpose	Important behavior
GET <code>/api/health</code>	Health check.	Returns <code>{status: ok}</code> .
POST <code>/api/sessions</code>	Create a UI session.	Returns a unique <code>thread_id</code> such as <code>ui-</code> . This <code>thread_id</code> maps frontend interactions to one LangGraph run.
GET <code>/api/runs/{thread_id}</code>	Read current run snapshot.	Uses <code>app.get_state</code> and pending interrupts to classify status as <code>idle</code> , <code>waiting_input</code> , or <code>completed</code> .
POST <code>/api/runs/start</code>	Start a research run.	Validates <code>StartResearchRequest</code> , invokes the graph with question, <code>user_id</code> , <code>max_iterations</code> , and empty messages.
POST <code>/api/runs/resume</code>	Resume a paused run.	Sends the user decision and optional human feedback into LangGraph using <code>Command(resume=...)</code> .
POST <code>/api/translate</code>	Translate final report text.	Uses <code>deep_translator.GoogleTranslator</code> and returns <code>translated_text</code> plus <code>target_language</code> .

The private helper `_snapshot_for_thread` is important because it adapts raw graph state into `RunSnapshotResponse`. It detects pending interrupts by reading `task.interrupts`, validates them as `HistoryInterruptModel` or `ReviewInterruptModel`, and includes current run artifacts for the UI.

3. LangGraph Flow

graph.py builds and compiles the StateGraph. The graph uses ChatOpenAI(model='gpt-4o-mini'), OpenAIEmbeddings(model='text-embedding-3-small'), Tavily, Wikipedia, MemorySaver, and InMemoryStore.

Main path

```
START -> initialize_run_node -> evaluate_guardrails_node -> load_history_node -> history_review_node ->
history_review_gate_node -> planner_node -> prepare_search_node -> tool_access_gate_node -> search_node
-> tools -> capture_tool_results_node -> reason_node -> evidence_selection_gate_node -> synthesise_node
-> review_gate_node -> publish_node -> save_history_node -> END
```

Conditional branches

Decision point	Possible routes
After guardrails	continue -> load_history_node; blocked -> guardrail_block_node -> END.
After history gate	proceed_with_context -> planner_node; start_fresh_plan -> planner_node; reuse_existing -> reuse_existing_report_node -> save_history_node -> END, but only when the latest exact question match exists.
After tool access gate	continue -> search_node; blocked -> guardrail_block_node -> END.
After search_node	If the LLM requested tools, route to ToolNode; otherwise go directly to reason_node.
After reason_node	CONTINUE -> prepare_search_node for another loop; DONE -> evidence_selection_gate_node when evidence exists, otherwise synthesise_node.
After evidence selection gate	User chooses one or more evidence items -> synthesise_node; no selection -> first evidence item is used as fallback.
After review_gate_node	approved -> publish_node; edited -> apply_edit_node -> publish_node; rejected -> planner_node.

Interrupts and resumability

- history_review_gate_node pauses when similar or related history exists and asks the user whether to proceed with context or start fresh; reuse_existing is offered only for the newest exact question match.
- evidence_selection_gate_node pauses before synthesis and asks the user to choose which normalized evidence items should drive the report.
- review_gate_node pauses before publishing and asks the user to approve, edit with feedback, or reject the draft.
- MemorySaver stores checkpoints per thread_id, so a later resume request can continue from the paused point.
- InMemoryStore stores shared research history while the backend process is alive; history_store.py also persists it to data/research_history.json.

4. What nodes.py Does

nodes.py is the workflow engine. It contains small graph nodes plus helper functions for cleaning inputs, scoring history, normalizing tool output, deduplicating/reranking evidence, gating evidence selection, computing metrics, validating drafts, and saving history.

Node	Role
initialize_run_node	Initializes default state keys such as iteration, decisions, reports, feedback, and metrics.
evaluate_guardrails_node	Sanitizes and assesses the question. Combines deterministic checks with structured LLM guardrail output.
guardrail_block_node	Creates a final blocked report when a request violates guardrail policy or no allowed tools are available.

Node	Role
load_history_node	Loads shared in-memory history and persisted JSON history, merges records, sorts newest first, and writes them into state.
history_review_node	Scores relevant prior records, asks the LLM whether the current question is similar, related, or new, and prepares history context.
history_review_gate_node	Raises a LangGraph interrupt when relevant history should be reviewed by the user.
reuse_existing_report_node	Uses the newest exact-match prior report as final_report without doing new search.
planner_node	Creates 2 to 3 targeted search directions, optionally using relevant history as context.
prepare_search_node	Increments the loop iteration counter before a search round.
capture_tool_results_node	Reads ToolMessage outputs, normalizes Tavily/Wikipedia results, deduplicates evidence, and updates metrics.
reason_node	Decides whether the graph has enough evidence or should continue another search loop.
evidence_selection_gate_node	Pauses the graph so the user can choose which evidence items should feed the report.
synthesise_node	Builds a structured DraftReportModel from selected evidence and history context.
review_gate_node	Raises the draft-review interrupt before publish.
apply_edit_node	Applies reviewer feedback to the draft summary before publishing. If comments accompany approval, the backend treats it as edited.
publish_node	Turns the draft into a polished FinalReportModel with published_report text.
save_history_node	Persists completed final reports into shared store and data/research_history.json unless the run reused history.

5. State and Schemas

state.py defines the internal LangGraph state. schemas.py defines strict Pydantic models for HTTP payloads, graph snapshots, interrupts, and structured LLM outputs.

Concept	Defined in	How it is used
ResearchState	state.py	Shared dictionary carried through every LangGraph node. It includes question, user_id, messages, iteration counters, history, evidence, selected evidence, draft_report, final_report, and metrics.
messages	state.py	Annotated with add_messages so LangGraph appends messages across node updates.
Guardrail models	schemas.py	Constrain guardrail status, action, risk flags, allowed tools, explanation, and clarification prompt.
DraftReportModel and FinalReportModel	schemas.py	Validate generated draft/final reports and prevent malformed report structures from reaching the UI.
HistoryReviewModel	schemas.py	Constrains the LLM's memory comparison output to match_type, rationale, and relevant history references.
Interrupt models	schemas.py	HistoryInterruptModel, EvidenceSelectionInterruptModel, and ReviewInterruptModel define the exact payload the frontend receives when the graph pauses.
RunSnapshotResponse	schemas.py	The main response shape returned by start, resume, and get snapshot endpoints.

Important state fields

- guardrails controls whether the run may proceed and which tools are allowed.
- past_topics holds loaded prior reports for history review.
- history_review and history_decision control whether prior work is reused, used as context, or ignored.
- retrieval_context stores selected history chunks; search_results stores normalized external evidence; selected_evidence stores the items chosen by the user for synthesis.
- research_plan guides tool queries; iteration and max_iterations control the search loop.
- draft_report, review_decision, human_feedback, and final_report represent the report lifecycle.

6. File-by-File Guide

File	Purpose
api.py	FastAPI app, CORS, HTTP endpoints, graph snapshot conversion, and translation endpoint.
app.py	Small adapter around LangGraph: create app, build config, build initial state, start/resume runs, inspect state and interrupts.
graph.py	Compiles the LangGraph StateGraph, creates LLM/embeddings/tools, wires every node and conditional route.
nodes.py	All workflow node implementations and helper logic for guardrails, history, evidence, reranking, evidence selection, reasoning, synthesis, review, publish, and persistence.
state.py	TypedDict definitions for internal graph state and nested report/evidence/history structures.
schemas.py	Pydantic validation for API requests/responses, LLM structured outputs, reports, interrupts, and snapshots.
prompts.py	Central prompt strings for planner, guardrails, history review, reasoner, synthesis, and publishing.

File	Purpose
tools.py	Creates TavilySearch, WikipediaQueryRun, weather, news, politics, and sports tools consumed by graph.py's ToolNode.
history_store.py	Loads, merges, sorts, and atomically saves JSON history in data/research_history.json.
validate_scenarios.py	HTTP-based validation runner that creates sessions, starts/resumes runs, and checks expected behavior.
sample_queries.py	Manual test query set for similar, related, and new history behavior.
demo.py	Minimal command-line graph demo with one canned research run and interrupt handling.
frontend.py	Optional Streamlit UI. The main frontend is react-frontend/.

7. Code-Level Interaction Map

This section explains how the files cooperate at runtime. LangGraph nodes do not mutate shared objects directly; each node returns a partial state update, and LangGraph merges that update into the current ResearchState for the active thread_id. That is why state.py, schemas.py, graph.py, nodes.py, app.py, and api.py have to stay in sync.

File	Reads	Writes	Why it matters
state.py	All graph inputs and outputs as TypedDict fields.	The state contract itself.	Defines what every node may read and update, including search_results, selected_evidence, draft_report, and final_report.
schemas.py	Incoming HTTP payloads and structured model outputs.	Validated request/response objects and interrupt payload models.	Keeps the frontend, API, and LangGraph checkpoints compatible.
graph.py	Current state plus compiled tool list.	Graph wiring, conditional edges, and node registration.	Decides when to search, when to pause, and which node runs next.
nodes.py	State plus prompts, history, and tool outputs.	Partial state updates for guardrails, history review, evidence selection, synthesis, review, publish, and persistence.	Implements the actual business logic of the workflow.
app.py	Thread config and current graph instance.	Command(resume=...) payloads or initial state payloads.	Provides the start/resume boundary between API code and LangGraph.
api.py	HTTP requests and current graph state.	RunSnapshotResponse objects and translation responses.	Turns graph state into the REST responses the UI consumes.

Runtime behavior

- api.py receives a request and validates it with schemas.py before calling app.py.
- app.py builds the thread config and calls the compiled LangGraph application created in graph.py.
- graph.py sends the current state into the selected node and decides the next node from the returned state update.
- nodes.py returns only the fields it changed. For example, capture_tool_results_node writes search_results and run_metrics, while review_gate_node writes review_decision and optional human_feedback.
- schemas.py keeps the interrupt payloads and snapshots strict so the frontend can render them without guessing at shape.

Tool selection flow

- nodes.py first infers an allowed tool set from the sanitized question. Every run always allows tavily and wikipedia, then adds weather, news, politics, or sports when the question clearly matches those domains.
- graph.py filters the concrete LangChain tool objects down to that allowed set before the model can call anything.
- The search node binds the LLM to only the allowed tools, so the model can only choose from the query-appropriate options.
- If the model asks for a tool, ToolNode executes it; if not, the graph advances to reason_node and eventually to synthesis.
- The prompts reinforce the preference: wikipedia for narrow background facts, tavily for broad live-web research, and weather/news/politics/sports for those specific query types.

Evidence selection at code level

- `reason_node` decides whether the graph is DONE or needs another search loop.
- When DONE and `search_results` exist, `graph.py` routes to `evidence_selection_gate_node` instead of `synthesise_node`.
- `evidence_selection_gate_node` interrupts the run and sends the top evidence items plus selection instructions to the frontend.
- The frontend sends `selected_evidence_ids` back through `/api/runs/resume`.
- `app.py` wraps those ids inside `Command(resume=...)` and the graph continues with `synthesise_node` using `selected_evidence` instead of the full board.

8. History and Fresh Search Behavior

Published reports are saved as records containing question, report, user_id, and created_at. The backend keeps a shared in-memory store and also persists records to data/research_history.json. When a run starts, load_history_node merges both sources and sorts records newest first.

- merge_history_records deduplicates by question plus created_at.
- sort_history_records makes the newest saved record win when the same question appears multiple times.
- history_review_node filters history by deterministic relevance before the LLM sees prior records.
- If no relevant record passes the threshold, the graph treats the question as new.
- The user can choose reuse_existing only when the newest exact question match exists; otherwise the interrupt allows proceed_with_context or start_fresh_plan.

9. Backend Data Flow Example

Step	Input	Output
1. Start	thread_id, question, user_id, max_iterations	Initial ResearchState in LangGraph.
2. Guardrails	question	Sanitized question, risk flags, allowed tools, proceed/revise/block decision.
3. History	question plus past_topics	new/similar/related assessment and optional interrupt.
4. Planning	question, guardrails, relevant history	2 to 3 search queries.
5. Search loop	plan, previous results, allowed tools	Tool calls, normalized evidence, metrics, reasoner DONE/CONTINUE.
6. Evidence selection	normalized evidence board	User-chosen evidence ids stored in selected_evidence_ids and selected_evidence.
7. Synthesis	selected evidence and history context	Structured draft report.
8. Human review	draft report	approved, edited, or rejected decision.
9. Publish	approved draft	FinalReportModel and polished published_report text.
10. Save	final report	History record in memory and JSON for future comparisons.

10. Developer Notes

- Environment variables: OPENAI_API_KEY is required for ChatOpenAI and embeddings. TAVILY_API_KEY is required for Tavily search.
- Backend command: uvicorn api:app --reload.
- Frontend command from react-frontend/: npm run dev.
- Validation command: python validate_scenarios.py --base-url http://localhost:8000/api.
- When debugging stale history, inspect data/research_history.json and the history_review interrupt payload returned by /api/runs/{thread_id}.
- When debugging search quality, inspect search_results, selected_evidence, retrieval_context, run_metrics, and the allowed_tools selected by guardrails.

11. Worked End-to-End Examples

These examples show how the backend behaves for common request types. They are intentionally written at the code-flow level so it is clear which file and node make each decision.

Example A - broad research query

- Question: "How should a company evaluate open-source AI coding assistants for enterprise use?"
- nodes.py keeps tavily and wikipedia available because the request is broad and factual.
- graph.py filters the tool list to the allowed set, and the search LLM may choose tavily first to gather live web evidence.
- After enough evidence is collected, reason_node returns DONE, evidence_selection_gate_node pauses the run, and the user chooses the strongest sources before synthesis.
- The final report is written by publish_node and saved by save_history_node for future reuse checks.

Example B - weather query

- Question: "What is the weather forecast for Chennai this week?"
- nodes.py recognizes the weather keyword and adds weather to the allowed tool set.
- The search node can now call the weather tool directly through ToolNode instead of forcing a general web search.
- The answer path is shorter because weather data usually satisfies the question without many retrieval loops.
- The published report still follows the same review and save steps as any other run.

Example C - exact history reuse

- Question: "What is the weather forecast for Chennai this week?" asked again after a report was already published.
- history_review_node compares the current question against stored history and finds the newest exact match.
- history_review_gate_node exposes reuse_allowed=true, so the frontend shows Reuse exact match in addition to Use as context and Start fresh.
- If the user chooses reuse_existing, reuse_existing_report_node copies the newest matching final report into final_report without running another search loop.
- This is the safe path for repeated questions because it reuses the last published answer instead of a loosely related older topic.

Example D - replan after rejection

- The user reviews the draft, adds comments, and clicks Reject.
- review_gate_node stores review_decision='rejected' and human_feedback in state.
- planner_node sees the rejection, resets the plan, and ignores prior search results when building a fresh plan.
- The workflow returns to the search loop with a new direction while keeping the same thread_id and run history.
- This keeps the final report anchored to the latest reviewer guidance instead of the previous draft.

12. API Payload Examples

The examples below are simplified so the shape is easy to read. The actual requests and responses are validated by schemas.py and by the FastAPI endpoints in api.py.

Start request

```
{ "thread_id": "ui-1234", "question": "How should we evaluate open-source AI coding assistants for enterprise use?", "user_id": "analyst-1", "max_iterations": 3 }
```

History interrupt response

```
{ "action": "review_history_match", "current_question": "What is the weather forecast for Chennai this week?", "match_type": "similar", "rationale": "A previously published report asks the same weather question.", "reuse_allowed": true, "reuse_candidate": { "question": "What is the weather forecast for Chennai this week?", "title": "Chennai Weekly Weather Report", "summary": "Warm conditions with scattered rain late in the week.", "user_id": "analyst-1", "created_at": "2026-06-29T08:00:00Z" } }
```

Evidence selection interrupt response

```
{ "action": "select_evidence_for_report", "question": "How should a company evaluate open-source AI coding assistants for enterprise use?", "research_plan": ["Compare licensing and data handling", "Review integration and security controls"], "current_evidence": [ {"chunk_id": "tavily-1", "title": "Vendor comparison article", "score": 0.83}, {"chunk_id": "wiki-1", "title": "Open-source software licensing", "score": 0.71} ], "instructions": "Select one or more evidence items to use for the report." }
```

Resume request for evidence selection

```
{ "thread_id": "ui-1234", "decision": "selected_evidence", "selected_evidence_ids": ["tavily-1", "wiki-1"], "human_feedback": "" }
```

13. What to Inspect When Debugging

- If the request is blocked too early, inspect `evaluate_guardrails_node`, the sanitized question, and the `allowed_tools` list in the guardrail state.
- If the wrong tool is being used, inspect `nodes.py _infer_allowed_tools`, `graph.py _allowed_tools_for_state`, and the prompt guidance in `prompts.py`.
- If old history is being reused incorrectly, inspect `_find_newest_exact_history_match`, `history_review_node`, and the interrupt payload returned by `history_review_gate_node`.
- If the report ignores selected evidence, inspect `selected_evidence_ids` in the resume request and `selected_evidence` in the run snapshot.
- If translation seems automatic, check `api.py /api/translate` and the frontend action that triggers it; it uses a translation package, not the report-generation LLM.